COMPUTER PROGRAMMING
CS-115

HOSPITAL MANAGEMENT SYSTEM

YUSRA BATOOL ZAIDI

EIFA SIDDIQUI

RAMEEN ZULFI

INSTRUCTORS:

MS HAMEEZA & MS MAHNOOR

Contents

TEAM CONTRIBUTIONS:

This project was developed through the combined efforts of three team members, each handling specific modules and responsibilities to ensure smooth functionality and proper documentation. The contributions are detailed below:

Rameen Zulfi

Rameen was responsible for building the overall user interaction flow of the system. This included designing and implementing the Main Menu, the Display Menu, and ensuring that all modules connected properly through the central program. The contributions include:

- Designing and implementing the main menu options: Patient, Doctor, Appointment, Billing and Exit.
- Creating interface-related functions for input/output display.
- Handling user input validation to prevent invalid entries.
- Integrating modules developed by other team members into one unified program.
- Managing the display layout of search results.
- Implementing the display and exit functionalities.
- Writing Chapter 1 – Introduction for the project report.

Eifa Siddiqui

Eifa contributed by building the core data-handling backbone of the system. This included struct design, CRUD operations, file management, and appointment scheduling. The contributions include:

- Creating structured definitions for Patient, Doctor, and Appointment.
- Implementing CRUD operations for patients, doctors and appointments.
- Developing robust file-handling utilities (save to file, read from file, update and delete operations).
- Ensuring all data is stored and retrieved correctly using .txt files.
- Implementing the linking functionality between various modules that are patients with doctors and appointments.
- Writing Chapter 2 – Methodology and Chapter 3 – Tools & Platforms for the final report.
- Formatting the complete MS Word report (title page, table of contents, layout, and references).

Yusra Batool Zaidi

Yusra handled the financial and verification components of the system, ensuring accurate billing and proper testing. Her contributions include:

- Helped in research, organization of the project work and ensured that all sections of the report were completed on time.
- Designing and implementing the Billing Module, including cost calculation and saving billing records.
- Conducting file handling system of the billing module.
- Creating a link between the doctors and billing module.
- Testing all major modules using at least five detailed test cases and capturing screenshots.
- Debugging file operations and verifying system stability.

- Writing Chapter 4 – Results and Chapter 5 – Conclusion & Future Work.

CHAPTER 01: INTRODUCTION

1.1 Introduction to the Problem

The Hospital Management System (HMS) aims to automate and simplify the daily operations of a hospital. It handles patient registration, doctor scheduling, appointments, and billing while reducing paperwork and human error. Its user-friendly, menu-driven interface allows staff to efficiently manage patient and doctor records. By centralizing information, the system ensures data accuracy, safety, and immediate accessibility.

1.2 Problem Description

Many hospitals still depend on manual record-keeping using paper files or registers. This approach is slow, prone to errors, and leads to misplacement of important data. Tasks like searching patient history, updating details, or checking doctor availability become time-consuming. Appointment management is also inefficient when handled manually.

The Hospital Management System solves these issues by providing fast data entry, easy searching, smooth record updating, and accurate billing, all stored digitally in one place. This reduces operational delays and improves overall efficiency.

1.3 Project Motivation, Relevance, and Beneficiaries

The motivation behind this project was to reduce the burden of manual workflows in hospitals and enable a more efficient system for storing and managing patient and doctor information. The system is especially relevant for small and medium healthcare facilities that need an organized structure but cannot afford complex software.

Primary Beneficiaries:

- **Hospital Administrators:** Manage records efficiently.
- **Doctors:** Access patient information instantly.
- **Patients:** Faster registration and reduced waiting time.
- **Billing Staff:** Generates accurate bills quickly without manual calculations.

1.4 Distinguishing Features of the Project

Key features that make this project unique include:

- A clear, structured, menu-driven interface.
- Multiple integrated modules (patient, doctor, appointments, billing).
- Input validation for preventing invalid entries.
- Permanent file-based storage for data retention.
- Concise bill display summarizing all key charges.

1.5 Most Challenging Part for you while working on the Project

The most challenging part was integrating all modules, patient, doctor, appointment, and billing, into one seamless system. Ensuring proper function communication, implementing input validation, maintaining menu flow, and preventing crashes from invalid input required careful debugging and planning. Maintaining clean user navigation was also a significant challenge.

1.6 New things Learnt in C while working on the project

During development, several new concepts were learned and applied:

- File handling using fopen(), fprintf(), fscanf(), and fclose().
- Using structures (struct) to store related data.
- Modular programming with separate functions for each feature.
- Input validation using loops and conditionals.
- Combining multiple C modules to create a scalable system.

CHAPTER 02: METHODOLOGY

2.1 How the project is developed?

The development process followed a structured approach:

1. Requirements Analysis

Identified all essential features: patient management, doctor management, appointments, billing, and file-based record storage.

2. System Structure Design

Designed the overall architecture, menu structure, data flow, and module interactions. Flowcharts and algorithm helped visualize system logic.

3. Data Structure Creation

Defined structure templates for storing patient and doctor information. Planned text file layouts for saving data.

4. Module Development

Coded each module separately using functions, ensuring input validation and error handling.

5. File Handling Implementation

Integrated file operations to permanently store and retrieve records.

6. Testing and Debugging

Tested using various inputs and corrected logic errors, file issues, and input-related crashes.

7. Final Integration & Output Formatting

Combined all modules into a single comprehensive system with neat formatting.

2.2 Design of the project with the help of algorithm and flow chart

Algorithm:

1. Initialize System

1. Start the program.
2. Initialize global arrays and counters:
3. Load existing data from files:
    - `patients.txt` → populate `patients[]` and `patientCount`
    - `doctors.txt` → populate `doctors[]` and `doctorCount`
    - `appointments.txt` → used when needed

2. Main Menu

1. Display main menu with options:
    - 1: Patient Module
    - 2: Doctor Module
    - 3: Appointment Module
    - 4: Billing Module
    - 5: Exit
2. Take user input for choice.
3. Depending on choice, go to the respective module

3. Patient Module

3.1 Menu Options

1. Add Patient
2. Update Patient
3. Search Patient
4. Delete Patient
5. Display All Patients
6. Exit

3.2 Add Patient

1. Ask the user to input a Patient ID.
2. Check if ID already exists (`checkDuplicateID`).
    - If duplicate, prompt again.
3. Input patient details: first name, last name, age, gender, disease.
4. Show the list of doctors related to disease, ask the user to choose one and assign the doctor automatically based on disease.
5. Concatenate full name (`firstName + lastName`).
6. Ask for confirmation to save.
7. Append data to `patients.txt`.

3.3 Update Patient

1. Input Patient ID to update.
2. Search for patients in `patients.txt`.
3. Display patient details.
4. Ask which field to update (name, age, gender, disease, doctor).
5. Update selected field(s).
6. Ask for confirmation to save changes.
7. Save all records to a temporary file → replace `patients.txt` with the temp file.

3.4 Search Patient

1. Choose to search by ID or Name.
2. If by ID → call `displayPatientByID`.
3. If by Name → search all patients whose `fullName`/`firstName`/`lastName` contains the input string.

3.5 Delete Patient

1. Input Patient ID to delete.
2. Display patient details.
3. Ask for confirmation to delete.
4. Copy all other records except the target patient to a temp file → replace `patients.txt` with temp file.

3.6 Display All Patients

1. Open `patients.txt`.
2. Read all patient records.
3. Display in tabular format with headings.

4. Doctor Module

4.1 Menu Options

1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit

4.2 Add Doctor

1. Input Doctor ID → check for duplicates.
2. Input doctor details: name, specialty, timings, fee, keywords.

3. Add doctor to `doctors[]`.
4. Save all doctors to `doctors.txt`.

4.3 List Doctors

- Loop through `doctors[]` and display summary (ID, Name, Specialty).

4.4 Show Doctor Details

- Input Doctor ID and display all details.

4.5 List Doctors by Specialty

- Input specialty string.
- Display all doctors whose specialty matches input (case-insensitive).

4.6 Search Doctor by Keyword

- Input disease/keyword string.
- Display all doctors whose `keywords` contain the input.

4.7 Update Doctor

1. Input Doctor ID to update.
2. Show doctor details.
3. Ask the user which field to update.
4. Update field(s) and save to `doctors.txt`.

4.8 Delete Doctor

1. Input Doctor ID to delete.
2. Show doctor details.
3. Ask for confirmation.
4. Remove doctor from `doctors[]` → save all to `doctors.txt`.

5. Appointment Module

5.1 Menu Options

1. Book Appointment
2. List All Appointments
3. Cancel Appointment
4. Exit

5.2 Book Appointment

1. Input Patient ID.
2. Validate patients exist in `patients[]`.

3. Show patient details.
4. Find related doctors:
    - Check which doctors have keywords matching a patient's disease.
5. Display available doctors.
6. Input Doctor ID.
7. Input appointment time.
8. Save appointments to `appointments.txt`.

5.3 List All Appointments

1. Read all records from `appointments.txt`.
2. Display in readable format:
    - Patient Name
    - Doctor Name
    - Appointment Time

6. Billing Module

6.1 Generate Bill

1. Input Patient ID.
2. Load patient details → populate global variables for doctor name, fee, patient name.
3. Input billing staff name.
4. Input room type:
    - Map to room charges per day.
5. Input number of days.
6. Input lab test type → calculate lab fees.
7. Calculate subtotal = (roomCharge × days) + doctorFee + testFee.
8. Apply tax (5%) and discount (fixed 500).
9. Calculate total = subtotal + tax − discount.
10. Input payment method (Cash/Card).
11. If cash → input amount received → calculate change.
12. Display formatted bill with all details.

7. Exit System

1. Save any unsaved data (if applicable).
2. Exit program.

Design of the Project:

The Hospital Management System (HMS) is designed to provide an organized, modular, and user-friendly interface to manage patients, doctors, appointments, and billing. The design ensures data consistency, reduces redundancy, and allows for easy future updates. The system is structured into four main modules, each handling specific responsibilities.

1. System Initialization

At startup, the system initializes all required variables, arrays, and counters. Existing data is loaded from the respective text files:

- patients.txt → Populates the array of patients and updates the patient count.
- doctors.txt → Populates the array of doctors and updates the doctor count.
- appointments.txt → Used when handling appointment booking and management.

This design allows the system to maintain persistent data between sessions and ensures that all operations are performed on up-to-date information.

2. Main Menu

The main menu acts as the central hub of the system. It presents the user with the following options:

1. Patient Module
2. Doctor Module
3. Appointment Module
4. Billing Module
5. Exit

The system takes the user's input and routes it to the corresponding module. This modular design ensures separation of concerns, so each module can operate independently and maintain clean, organized code.

3. Patient Module

The Patient Module is responsible for managing all patient-related information. The module supports the following operations:

3.1 Add Patient

- Users are prompted to enter patient details.
- The system checks for duplicate IDs to maintain data integrity.
- Based on the patient's disease, a list of relevant doctors is displayed for automatic assignment.
- Patient data, including assigned doctor, is saved to patients.txt.

3.2 Update Patient

- Users select a patient by ID.

- The system displays patient details and allows updates to specific fields (name, age, gender, disease, doctor).
- Any update operation creates a temporary file to ensure atomic changes, replacing the original file only after confirmation.

3.3 Search Patient

- Supports search by **ID** or **Name**.
- Search by name is flexible, matching any part of the patient's first, last, or full name.

3.4 Delete Patient

- Deletes patient records after confirmation.
- Uses a temporary file to maintain data integrity during deletion.

3.5 Display All Patients

- Displays all patient records in a tabular format with proper headings.

4. Doctor Module

The Doctor Module manages doctor information and ensures that patient-doctor assignments are accurate. Features include:

- Add, list, update, or delete doctors.
- Display doctors by specialty or search using disease-related keywords.
- This module allows the system to dynamically match patients to doctors based on expertise.

5. Appointment Module

The Appointment Module handles scheduling and management of appointments:

- Book appointments for patients by matching their disease with relevant doctors.
- Display all existing appointments.
- Cancel appointments with confirmation.

The system validates both patient and doctor information before saving appointment details to ensure consistency.

6. Billing Module

The Billing Module automates bill generation based on:

- Doctor fees
- Room type charges
- Laboratory test charges
- Tax and discounts
- Payment method handling (Cash or Card)

The billing process calculates subtotals, tax, discounts, and final payment, producing a neatly formatted bill for patients.

7. File-Based Data Management

All data is stored in text files, allowing persistent storage:

- patients.txt stores patient records.
- doctors.txt stores doctor records.
- appointments.txt stores appointment details.

Temporary files are used during update or delete operations to prevent data corruption.

8. Modular and User-Friendly Design

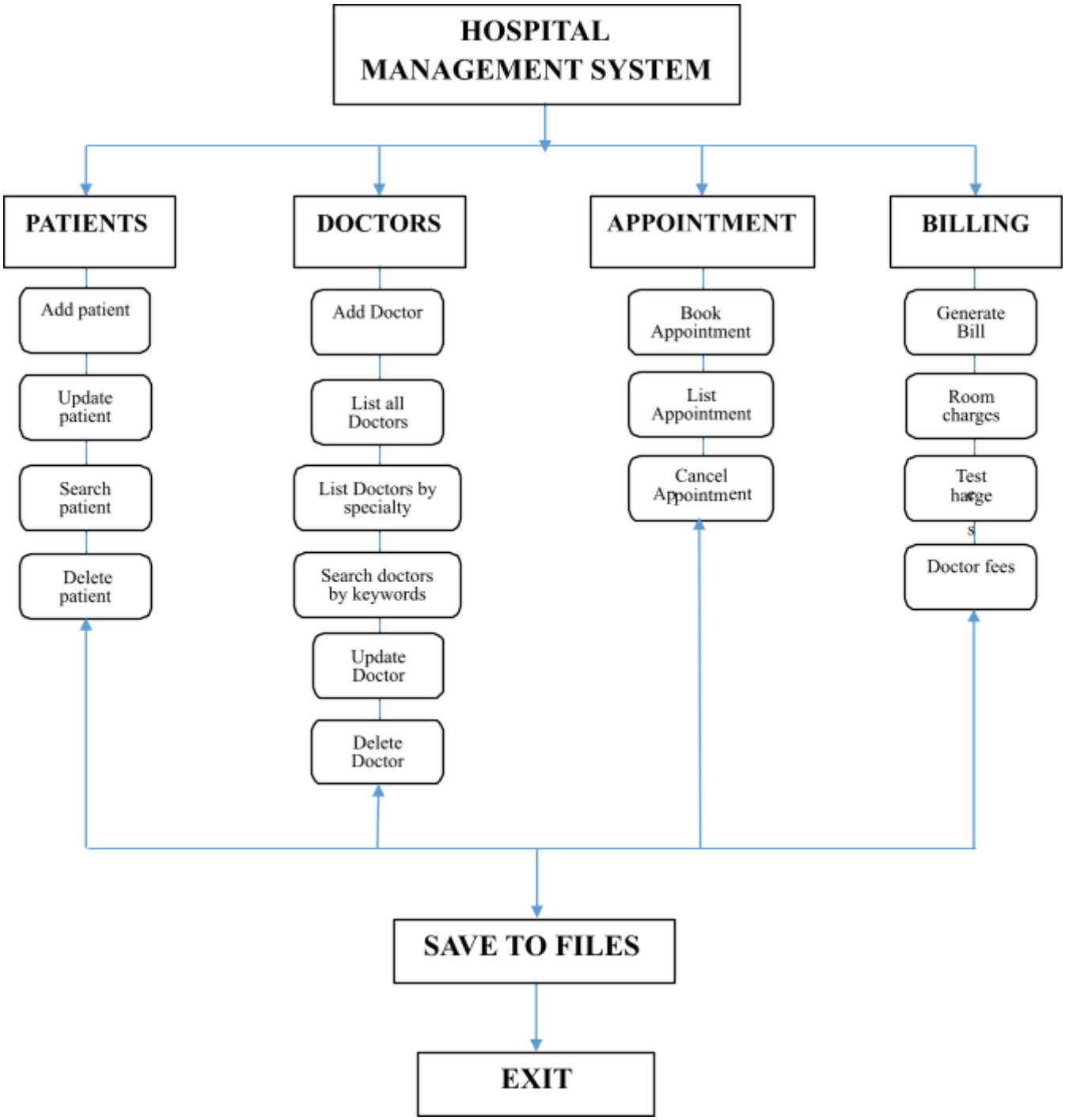
- Each module is self-contained, which simplifies maintenance and testing.
- The program uses functions for repetitive tasks like checking duplicate IDs, displaying records, or searching by keyword.
- User prompts, confirmations, and error handling ensure a smooth user experience.

9. Flow of Operations

1. Start program → load data → display main menu.
2. The user chooses module → module performs required operation.
3. The patient module can call doctor search for doctor assignment.
4. Appointment module cross-references patient and doctor records.
5. Billing module calculates costs based on stored records.
6. Exit → all unsaved data is written to files.

This design ensures **data integrity, ease of navigation, and scalability** for future enhancements, such as adding new modules or integrating a database.

Flowchart:


HOSPITAL MANAGEMENT SYSTEM
PATIENTS
Add patient
Update patient
Search patient
Delete patient
DOCTORS
Add Doctor
List all Doctors
List Doctors by specialty
Search doctors by keywords
Update Doctor
Delete Doctor
APPOINTMENT
Book Appointment
List Appointment
Cancel Appointment
BILLING
Generate Bill
Room charges
Test charges
Doctor fees
SAVE TO FILES
EXIT

CHAPTER 03: TOOLS AND PLATFORMS

3.1 Tools and Platforms Used

- **Programming Language:** C
- **Application software (IDE):** VS Code
- **Compiler:** Dev-C++
- **Data Storage:** Text files (.txt)
- **Operating System:** Windows 10
- **Libraries Used:** stdio.h, string.h, ctype.h
- **Testing:** Manual testing with varied input values

CHAPTER 04: RESULTS

4.1 Test Cases and Results

1. Main Menu

Screenshots show various modules and the clean main function with only switch case.

SHOT 1:

```
C:\Users\DELL\OneDrive\Desktop\CP_PROJECT - Copy\Hospital Management System>gcc main.c patients.c doctors.c appointme
nts.c billing.c -o hospital_app

C:\Users\DELL\OneDrive\Desktop\CP_PROJECT - Copy\Hospital Management System>hospital_app

=======================================
   HOSPITAL MANAGEMENT SYSTEM MAIN MENU
=======================================
1. Patients Module
2. Doctors Module
3. Appointments Module
4. Billing Module
5. Exit
=======================================
Enter your choice: 1


Patient's Menu

1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 6
Exit

=======================================
   HOSPITAL MANAGEMENT SYSTEM MAIN MENU
=======================================
1. Patients Module
2. Doctors Module
3. Appointments Module
4. Billing Module
5. Exit
```

SHOT 2:

```
5. Exit
=======================================
Enter your choice: 2

--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Speciality
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit
Enter choice: 8
Exit

=======================================
   HOSPITAL MANAGEMENT SYSTEM MAIN MENU
=======================================
1. Patients Module
2. Doctors Module
3. Appointments Module
4. Billing Module
5. Exit
=======================================
Enter your choice: 3

--- Appointment Module ---
1. Book Appointment
2. List All Appointments
3. Exit to Main Menu
Enter choice: 3

=======================================
   HOSPITAL MANAGEMENT SYSTEM MAIN MENU
=======================================
1. Patients Module
```

SHOT 3:

```
=======================================
   HOSPITAL MANAGEMENT SYSTEM MAIN MENU
=======================================
1. Patients Module
2. Doctors Module
3. Appointments Module
4. Billing Module
5. Exit
=======================================
Enter your choice: 4

--- BILLING MODULE ---
Enter Patient ID: 21

Patient Found:
----------------------------
ID: 21
Name: muhammad saleh
Age: 22
Gender: male
Disease: heart
Doctor: james
----------------------------
Enter Billing Staff Name: harry

Select Room Type:
 1. General Ward (2000.00 per day)
 2. Semi-Private Room (3500.00 per day)
 3. Private Room (5000.00 per day)
 4. ICU (10000.00 per day)
Enter choice (1-4): 1
Enter Number of days stayed: 2

Select Lab Test:
 1. No Lab Test (Charges: 0.00)
 2. Blood Test (Charges: 500.00)
 3. X-Ray (Charges: 1000.00)
```

SHOT 4:

```
Select Lab Test:
 1. No Lab Test (Charges: 0.00)
 2. Blood Test (Charges: 500.00)
 3. X-Ray (Charges: 1000.00)
 4. MRI (Charges: 5000.00)
 5. CT Scan (Charges: 7000.00)
Enter choice (1-5): 2
Enter number of such test done: 1

Doctor's Fees: 6000.00
Subtotal (before tax and discount): 10500.00

Enter Payment Method (C for Cash / K for Card): k

----------------------------------------
 HOSPITAL BILL
----------------------------------------
Patient ID: 21
Patient Name: muhammad saleh
Doctor: james
Consultation Fee: 6000.00
----------------------------------------
Room Charges (2 days - General Ward): 4000.00
Lab Tests (Blood Test x 1): 500.00
----------------------------------------
Subtotal: 10500.00
Tax (5%): 525.00
Discount: 500.00
----------------------------------------
Total Bill: 10525.00
----------------------------------------
Payment Method: k
----------------------------------------
Thank you for choosing our hospital!
----------------------------------------
```

SHOT 5:

```
Discount: 500.00
----------------------------------------
Total Bill: 10525.00
----------------------------------------
Payment Method: k
----------------------------------------
Thank you for choosing our hospital!
----------------------------------------

========================================
   HOSPITAL MANAGEMENT SYSTEM MAIN MENU
========================================
1. Patients Module
2. Doctors Module
3. Appointments Module
4. Billing Module
5. Exit
========================================
Enter your choice: 5
Exiting System. Goodbye!

C:\Users\DELL\OneDrive\Desktop\CP_PROJECT - Copy\Hospital Management System>
```

2. Patient Module

Screenshots show adding, updating, searching, and deleting patient records. All data is stored in text files with confirmation messages.

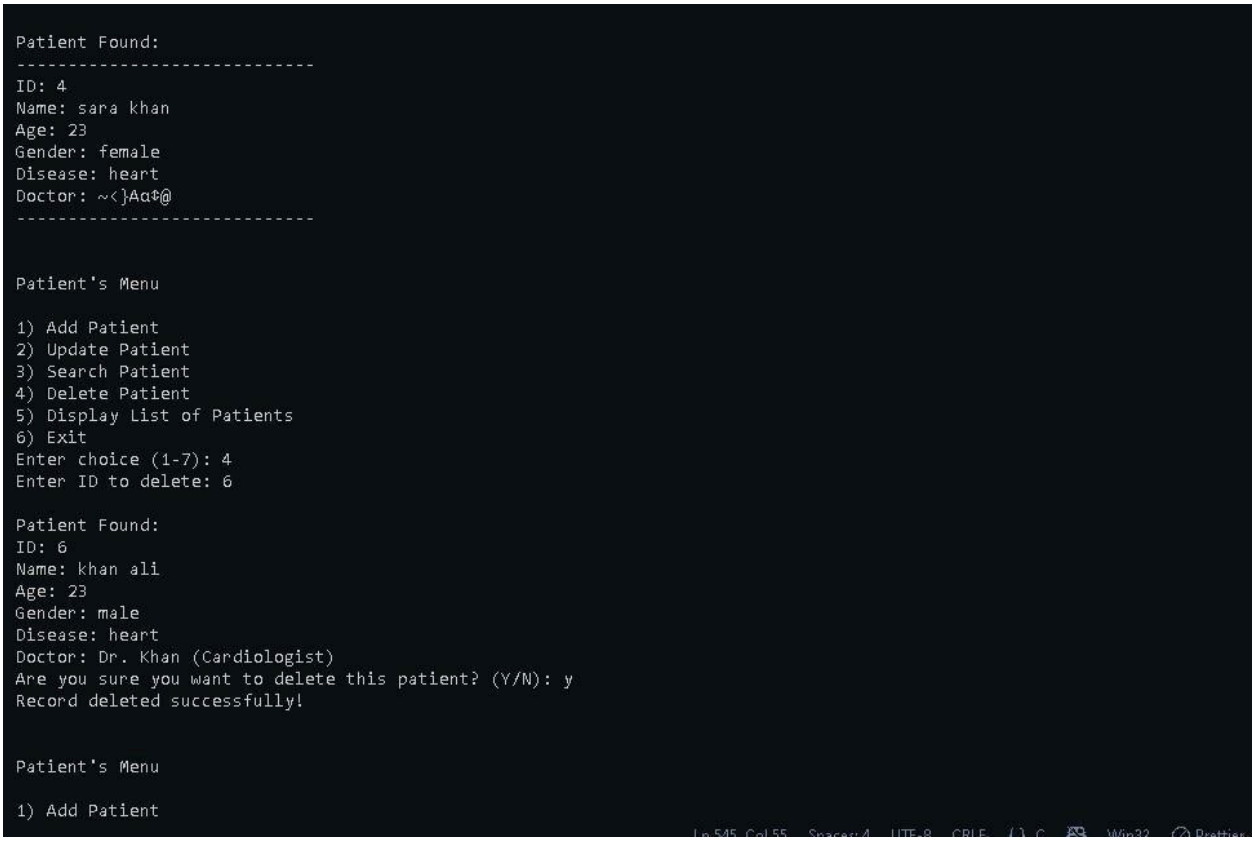
SHOT 1:

```
Patient's Menu

1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 1
Enter Patient ID: 10
First Name: Rafay
Last Name: Khan
Age: 43
Gender: Male
Disease: heart

Available Doctors for heart:
1) james  | cardiologist  |  9AM-5PM  |  6000   |  heart
Enter Doctor ID to assign: 1
Confirm Add (Y/N): y
Patient added successfully!


Patient's Menu

1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 2
Enter ID to update: 2

Patient Found:
----------------------------
```

SHOT 2:

```
Patient Found:
----------------------------
ID: 2
Name: sam andy
Age: 34
Gender: male
Disease: cough
Doctor: jerry
----------------------------

1.Name
2.Age
3.Gender
4.Disease
5.Doctor
6.Cancel
Choice: 2
New Age: 33
Save changes? (Y/N): y
Updated successfully!


Patient's Menu

1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 3

1. Search by ID
2. Search by Name
Enter choice: 1
Enter ID: 4
```

SHOT 3:


Patient Found:
ID: 4
Name: sara khan
Age: 23
Gender: female
Disease: heart
Doctor: ~<}Aα¢@
Patient's Menu
1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 4
Enter ID to delete: 6
Patient Found:
ID: 6
Name: khan ali
Age: 23
Gender: male
Disease: heart
Doctor: Dr. Khan (Cardiologist)
Are you sure you want to delete this patient? (Y/N): y
Record deleted successfully!
Patient's Menu
1) Add Patient

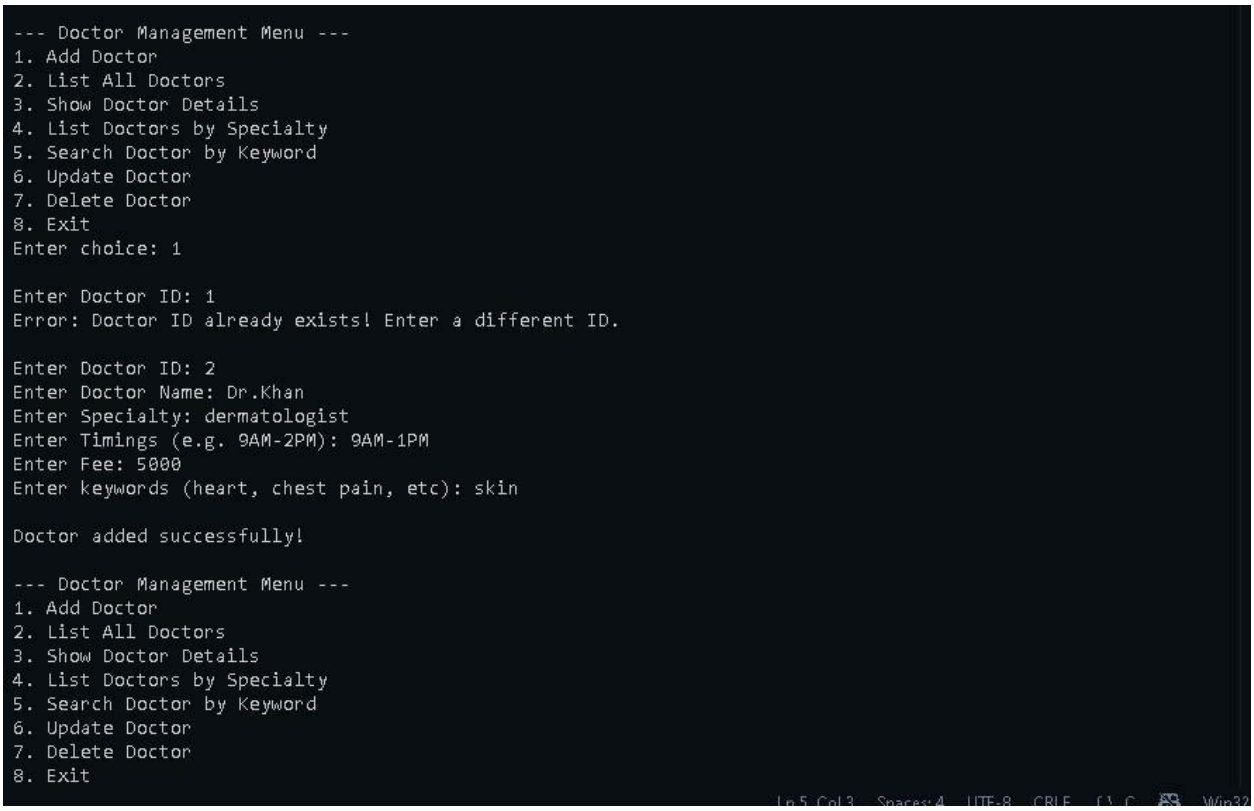
SHOT 4:

```
Patient's Menu

1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 5
ID      First Name      Last Name       Age     Gender  Disease
-------------------------------------------------------------
1       James           doe             23      male    malaria,jery
2       sam             andy            33      male    cough,jerry
3       sarah           adams           33      female  malaria,smith
5       Muhammad                Hamza           20      Male    heart,Dr. Khan (Cardiologist)
4       sara            khan            23      female  heart,~<}Aq↕@
8       hala            khan            21      female  heart,├♀■Ua↕@
9       Hala            khan            21      female  heart,james
10      Rafay           Khan            43      Male    heart,james


Patient's Menu

1) Add Patient
2) Update Patient
3) Search Patient
4) Delete Patient
5) Display List of Patients
6) Exit
Enter choice (1-7): 6
Exit
```

3. Doctor Module

Screens display doctor information management, including specialization and visiting hours.

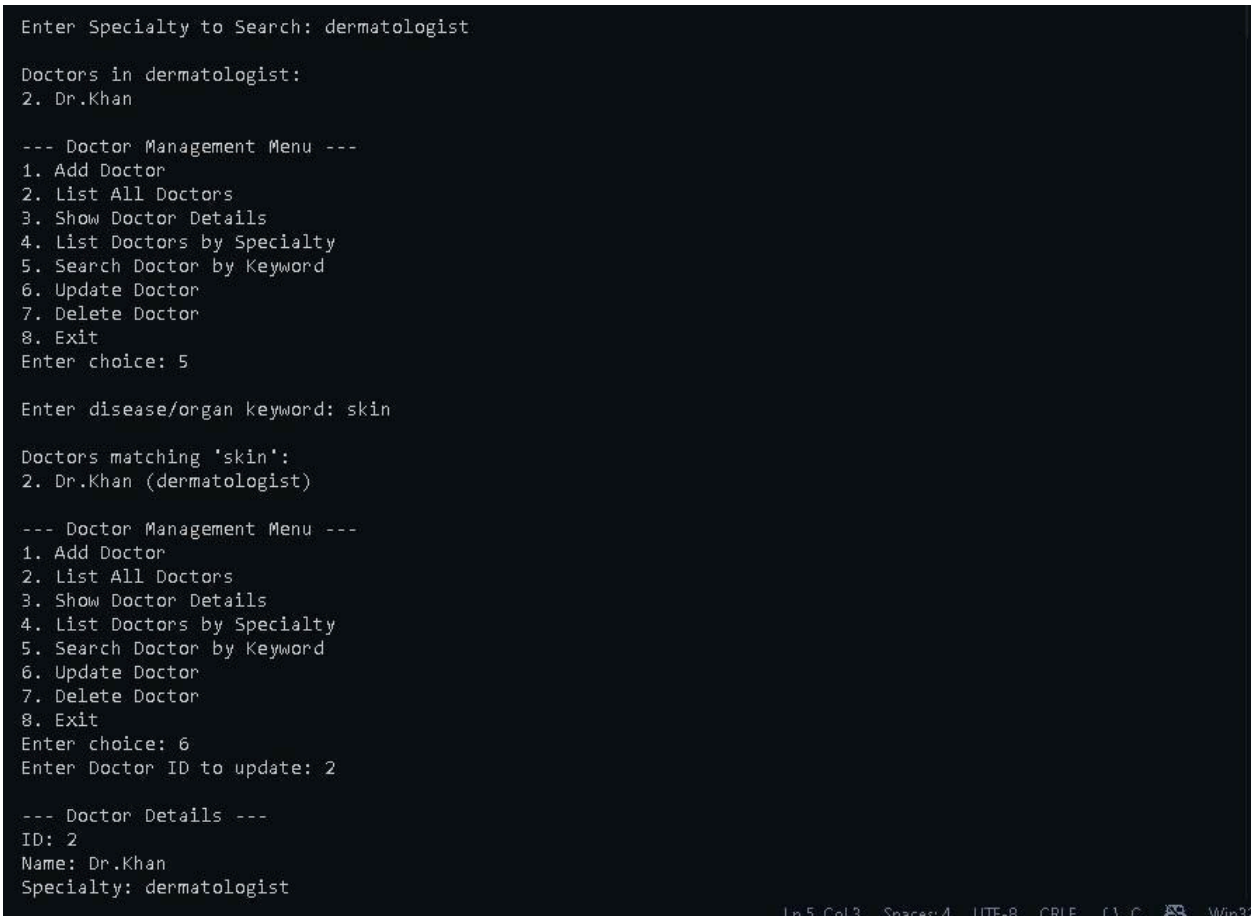
SHOT 1:


--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit
Enter choice: 1
Enter Doctor ID: 1
Error: Doctor ID already exists! Enter a different ID.
Enter Doctor ID: 2
Enter Doctor Name: Dr.Khan
Enter Specialty: dermatologist
Enter Timings (e.g. 9AM-2PM): 9AM-1PM
Enter Fee: 5000
Enter keywords (heart, chest pain, etc): skin
Doctor added successfully!
--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit

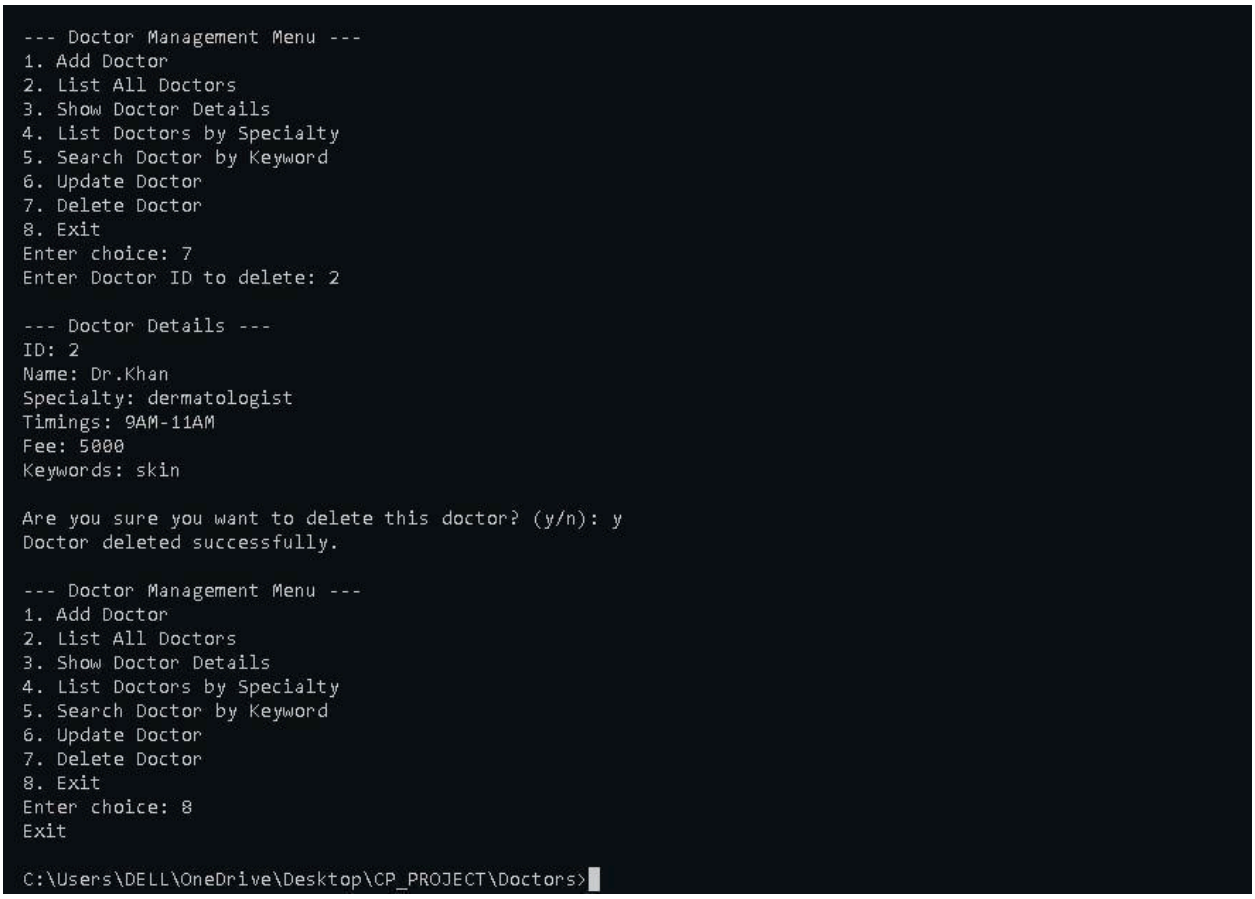
SHOT 2:

```
Enter choice: 2

--- List of Doctors ---
1. james (cardiologist)
2. Dr.Khan (dermatologist)

--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit
Enter choice: 3
Enter Doctor ID: 2

--- Doctor Details ---
ID: 2
Name: Dr.Khan
Specialty: dermatologist
Timings: 9AM-1PM
Fee: 5000
Keywords: skin

--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit
Enter choice: 4
```

SHOT 3:


Enter Specialty to Search: dermatologist
Doctors in dermatologist:
2. Dr.Khan
--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit
Enter choice: 5
Enter disease/organ keyword: skin
Doctors matching 'skin':
2. Dr.Khan (dermatologist)
--- Doctor Management Menu ---
1. Add Doctor
2. List All Doctors
3. Show Doctor Details
4. List Doctors by Specialty
5. Search Doctor by Keyword
6. Update Doctor
7. Delete Doctor
8. Exit
Enter choice: 6
Enter Doctor ID to update: 2
--- Doctor Details ---
ID: 2
Name: Dr.Khan
Specialty: dermatologist

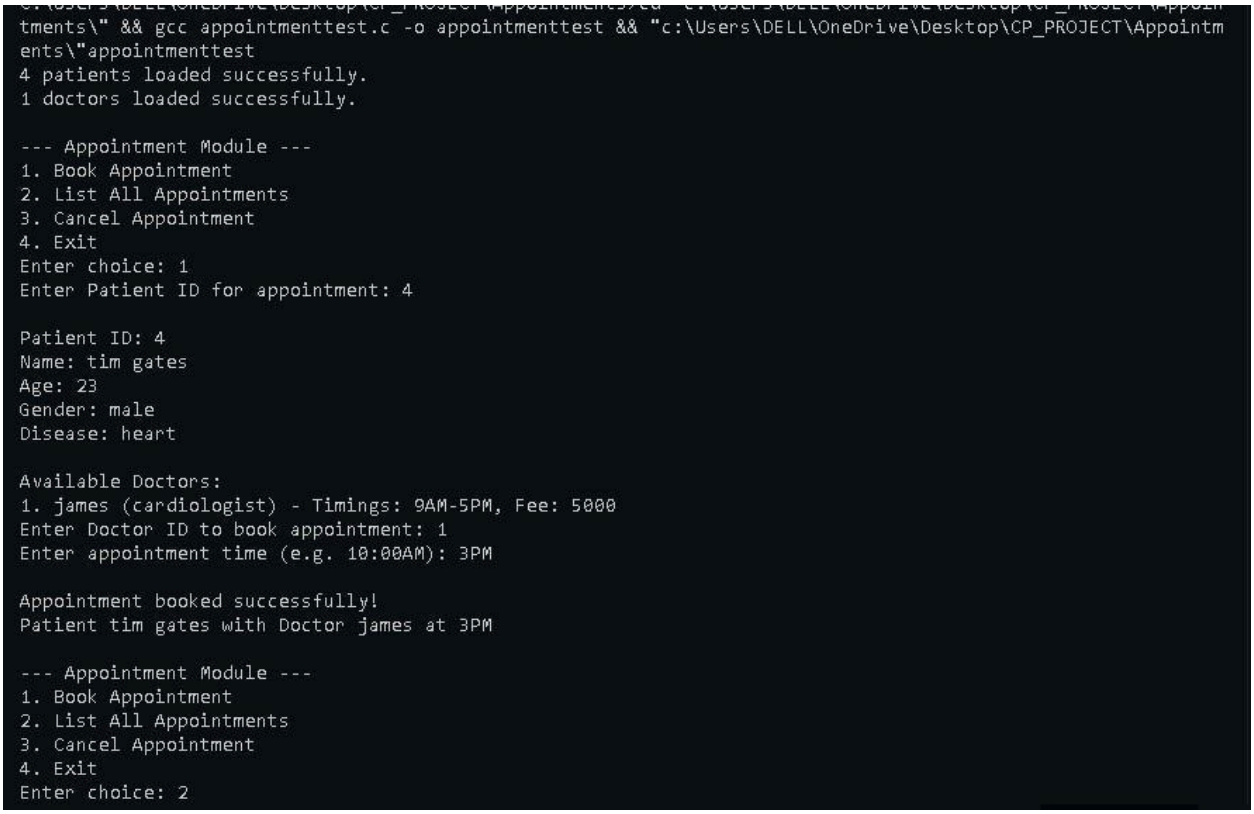
SHOT 4:



4. Appointment Module

Demonstrates scheduling or deleting appointments with proper patient-doctor matching.

SHOT 1:


tments\" && gcc appointmenttest.c -o appointmenttest && "c:\Users\DELL\OneDrive\Desktop\CP_PROJECT\Appointm
ents\"appointmenttest
4 patients loaded successfully.
1 doctors loaded successfully.
--- Appointment Module ---
1. Book Appointment
2. List All Appointments
3. Cancel Appointment
4. Exit
Enter choice: 1
Enter Patient ID for appointment: 4
Patient ID: 4
Name: tim gates
Age: 23
Gender: male
Disease: heart
Available Doctors:
1. james (cardiologist) - Timings: 9AM-5PM, Fee: 5000
Enter Doctor ID to book appointment: 1
Enter appointment time (e.g. 10:00AM): 3PM
Appointment booked successfully!
Patient tim gates with Doctor james at 3PM
--- Appointment Module ---
1. Book Appointment
2. List All Appointments
3. Cancel Appointment
4. Exit
Enter choice: 2

SHOT 2:

```
4. Exit
Enter choice: 2

--- Appointments ---
Patient: tim gates, Doctor: james, Time: 9:00AM
Patient: tim gates, Doctor: james, Time: 4:00PM
Patient: tim gates, Doctor: james, Time: 4PM
Patient: tim gates, Doctor: james, Time: 3PM

--- Appointment Module ---
1. Book Appointment
2. List All Appointments
3. Cancel Appointment
4. Exit
Enter choice: 3

--- Appointments ---
1. Patient: tim gates, Doctor: james, Time: 9:00AM
2. Patient: tim gates, Doctor: james, Time: 4:00PM
3. Patient: tim gates, Doctor: james, Time: 4PM
4. Patient: tim gates, Doctor: james, Time: 3PM
Enter appointment number to cancel: 2
Appointment canceled successfully!

--- Appointment Module ---
1. Book Appointment
2. List All Appointments
3. Cancel Appointment
4. Exit
Enter choice: 2

--- Appointments ---
Patient: tim gates, Doctor: james, Time: 9:00AM
Patient: tim gates, Doctor: james, Time: 4PM
Patient: tim gates, Doctor: james, Time: 3PM
```

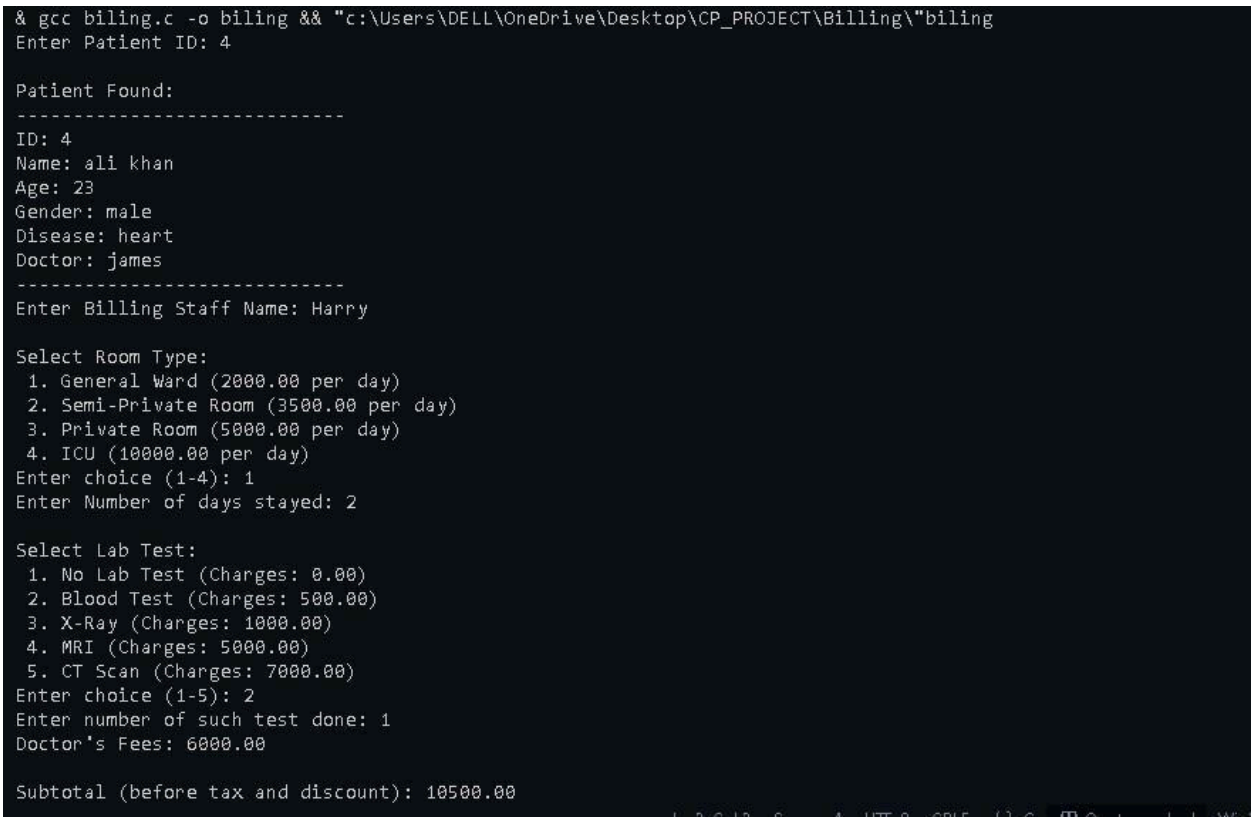
SHOT 3:

```
Enter choice: 2

--- Appointments ---
Patient: tim gates, Doctor: james, Time: 9:00AM
Patient: tim gates, Doctor: james, Time: 4PM
Patient: tim gates, Doctor: james, Time: 3PM

--- Appointment Module ---
1. Book Appointment
2. List All Appointments
3. Cancel Appointment
4. Exit
Enter choice: 4
Exit
```

5. Billing Module

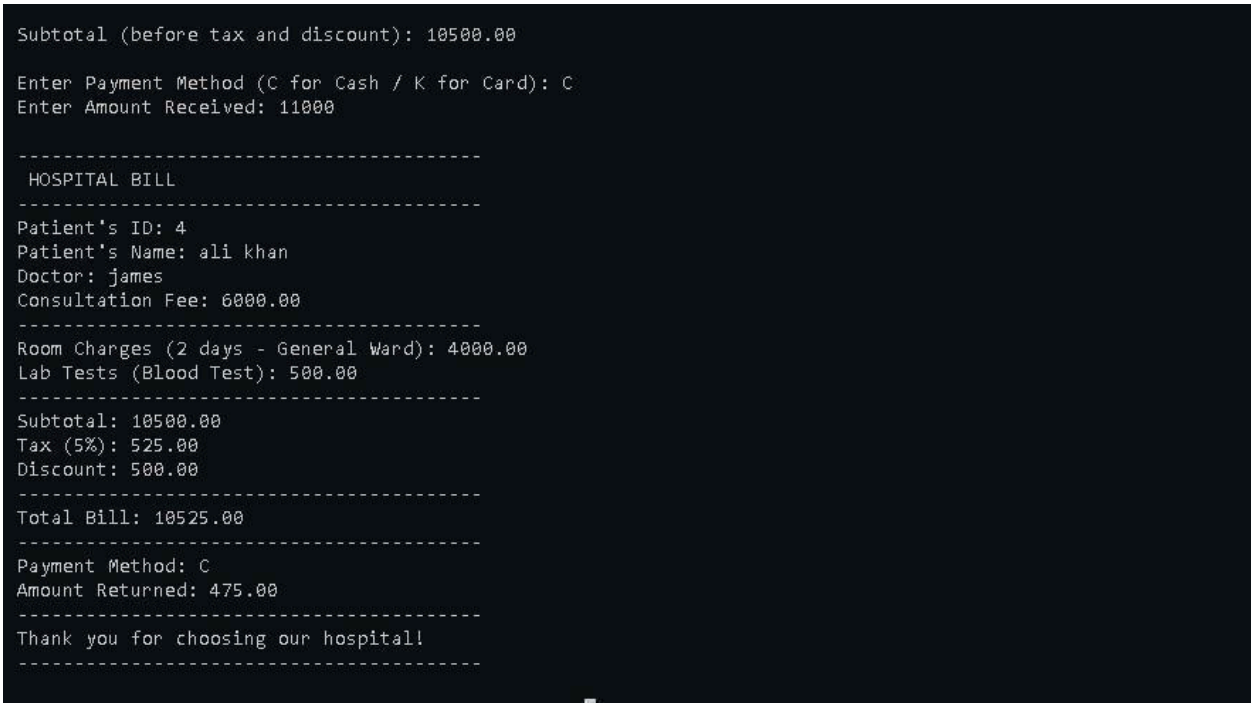
Shows calculation of total bill including consultation, room charges, and lab fees with discounts and tax applied.

SHOT 1:



```
& gcc biling.c -o biling && "c:\Users\DELL\OneDrive\Desktop\CP_PROJECT\Billing\"biling
Enter Patient ID: 4

Patient Found:
----------------------------
ID: 4
Name: ali khan
Age: 23
Gender: male
Disease: heart
Doctor: james
----------------------------
Enter Billing Staff Name: Harry

Select Room Type:
 1. General Ward (2000.00 per day)
 2. Semi-Private Room (3500.00 per day)
 3. Private Room (5000.00 per day)
 4. ICU (10000.00 per day)
Enter choice (1-4): 1
Enter Number of days stayed: 2

Select Lab Test:
 1. No Lab Test (Charges: 0.00)
 2. Blood Test (Charges: 500.00)
 3. X-Ray (Charges: 1000.00)
 4. MRI (Charges: 5000.00)
 5. CT Scan (Charges: 7000.00)
Enter choice (1-5): 2
Enter number of such test done: 1
Doctor's Fees: 6000.00

Subtotal (before tax and discount): 10500.00
```

SHOT 2:


Subtotal (before tax and discount): 10500.00
Enter Payment Method (C for Cash / K for Card): C
Enter Amount Received: 11000
HOSPITAL BILL
Patient's ID: 4
Patient's Name: ali khan
Doctor: james
Consultation Fee: 6000.00
Room Charges (2 days - General Ward): 4000.00
Lab Tests (Blood Test): 500.00
Subtotal: 10500.00
Tax (5%): 525.00
Discount: 500.00
Total Bill: 10525.00
Payment Method: C
Amount Returned: 475.00
Thank you for choosing our hospital!

CHAPTER 05: CONCLUSION AND FUTURE EXPANSION

5.1 Conclusion

The Hospital Management System successfully automates vital hospital operations. It reduces paperwork, minimizes human error, and enables fast retrieval and updating of records. Using C language concepts like structures, file handling, functions, and modular programming, the system meets essential needs of small and medium healthcare facilities.

It improves operational efficiency through:

- Organized patient record storage
- Easy searching and updating
- Proper doctor management
- Automated billing
- Reduced manual workload

This project strengthened our programming skills and enhanced our understanding of real-world system design.

5.2 Future Expansion

- **Graphical User Interface (GUI):** Convert to Java, C#, or Python GUI.
- **Database Integration:** Use MySQL instead of text files.
- **User Authentication:** Role-based login system for admin, staff, and doctors.
- **Use of Linked Lists:** For dynamic record management.
- **Appointment Conflict Prevention:** Ensure no double-booking of time slots.

REFERENCES

1. **Aga Khan University Hospital – Official Website**
   We reviewed their "Find a Doctor" section, where users can search for doctors using keywords, affected organs, and medical specialties. This helped us understand real-world hospital search workflows and inspired our doctor-search functionality.
2. **Paul Deitel & Harvey Deitel**, ***C How to Program***
   Concepts of C programming, structures, and file handling.